Borla

Project Documentation

Student: **George Boadu Junior**

Student ID: **22427354**

Course: CSCD 602 — Advanced Software Engineering, University of Ghana

Document: Project_Documentation.pdf · Version 1.0 · 13 August 2026

1. Project title

Borla — a real-time matchmaker connecting Ghanaian households with nearby roaming waste collectors. ("Borla" is Ghanaian slang for rubbish/waste.)

2. Problem statement

In many Ghanaian neighbourhoods, formal municipal waste collection is unreliable or absent, so households depend on informal, roaming waste collectors (tricycle/pickup operators) who work a patch of streets looking for waste to collect for a fee. There is no shared channel connecting the two sides: a household with waste ready has no way to signal it beyond hoping a collector happens to pass, and a collector has no way to see where waste is actually waiting without driving around speculatively. Both sides waste time and, for collectors, fuel/effort — a classic two-sided discovery problem that ride-hailing-style matchmaking solves in other domains.

3. Aim and objectives

Aim: build and deploy a functional, end-to-end matchmaking platform that lets a household announce or directly request a waste pickup, and lets a collector discover and respond to that demand in real time — while demonstrating disciplined requirements engineering, effort estimation, design, implementation, testing, technical-debt management, and deployment practice within a hard time constraint.

Objectives:

1. Model the matching problem as two independent mechanisms — a stateless broadcast and a stateful direct request — rather than one over-complicated dispatcher.
2. Build a working two-sided trust layer (ratings) so both parties have a reason to behave well.
3. Give operations staff (admin) real levers: verify collectors, moderate content, watch live density, and retune behaviour without a redeploy.
4. Apply an explicit, justified effort-estimation technique to *scope* the build, not just to describe it after the fact.
5. Ship something genuinely running at a public URL, not a mock-up — with real, executable automated tests and a maintained technical-debt register.

4. Stakeholders

Stakeholder	Interest
Households	Fast, low-effort way to get waste collected; trust that a collector who says they're coming actually will
Collectors	Reliable visibility into nearby demand; protection from wasted trips to stale/cleared pins; a fair way to build a reputation
Platform operators (Admin)	Ability to onboard/verify collectors, keep content safe, see whether the marketplace is healthy per neighbourhood
Course examiner	Evidence of systematic Advanced Software Engineering practice across the full lifecycle

5. Requirements analysis

Requirements were gathered by extending an existing, larger architectural vision (`borla-technical-design.md`, authored ahead of this build as the target end-state) down to what is actually implementable, deployable, and testable inside this exam's constraints. The extension/reduction process:

1. Took every functional area of the original design (two matching planes, presence, trust layer, admin, AI) as the candidate requirement pool.
2. Removed anything that depends on infrastructure or accounts that could not be provisioned inside the exam (native Android background location, a funded SMS gateway, a managed Redis instance, a second admin application) — two of those four were later resolved once real credentials became available mid-build: GiantSMS (`Technical_Debt_Plan.md` TD-02) and Upstash Redis (TD-05). Native Android and the separate admin app remain descoped.
3. Classified everything that remained with MoSCoW (`SRS.md` §6).
4. Ran Use Case Points estimation (§7 below) against the *remaining* scope to confirm the prioritisation was still necessary even after the initial cut — it was, by roughly 16–80× the nominal time-box, which is exactly the signal the exam brief anticipates.

Full functional/non-functional requirement tables are in `SRS.md` §4–§5.

6. Software Requirements Specification (summary)

See `SRS.md` for the complete document. In summary: three human actors (household, collector, admin) plus two system actors (the Gemini moderation API, the BullMQ scheduler); 26 functional requirements spanning auth, the broadcast plane, the request plane, the trust layer, and the admin console; ten non-functional requirements covering security, privacy, reliability, usability, and fail-safe moderation.

7. Software effort estimation (summary)

Technique: Use Case Points, cross-checked with expert top-down estimation. Full working in `SRS.md` §7. Headline numbers:

Method	Result	Interpretation
UCP (216 UUCP × 1.05 TCF × 0.845 EF × 20h/UCP)	**≈ 3,833 person-hours** (≈ 22.6 person-months)	Cost to build this exact scope to full commercial rigour
Expert top-down (module-by-module + 15% overhead)	**≈ 770 person-hours** (≈ 4.5 person-months solo)	Cost for one experienced engineer to build a lean-but-real MVP
Nominal exam window	48 hours	Neither estimate is remotely close to this — by design (see SRS §7.7)

How this shaped scope: since both estimates dwarf the available time regardless of how aggressively the *original* design was already cut, the estimate's job was to decide *how* to spend the tiny fraction of either number actually available: breadth (a complete, working, thin slice of every Must-Have mechanism) rather than depth (fully hardening a narrower subset). The resulting shortfall — against either estimate — is exactly what §12 (Technical debt) below catalogues.

8. System analysis

Actors and their goals: a household wants waste gone with minimum effort and no wasted trust; a collector wants reliable, current demand signals and protection from stale pins; an admin wants levers to keep the marketplace healthy and safe without needing a redeploy for every tuning change.

Core domain insight carried over from the original design: the two matching mechanisms have opposite integrity properties on purpose. A **broadcast** is stateless and lossy — no collector "owns" a pin, so there is no claim race to resolve, at the cost of possible wasted trips (mitigated by a strong clear-prompt and short TTL). A **request** is stateful and narrow — exactly one collector is targeted, so the only correctness hazard is a stale accept/reject arriving after the request already resolved, which is closed with a conditional `WHERE status IN (...)` update rather than a lock. Keeping these two mechanisms structurally separate (different tables, different state machines) is why neither had to inherit the other's complexity.

Key non-functional pressure: every mutation that touches trust (reviews, contact reveal, admin actions) needed an explicit integrity rule *enforced in the schema*, not just in application code — see the `UNIQUE(author_id, request_id)` / `UNIQUE(author_id, broadcast_id)` constraints and the `CHECK` that a review references exactly one interaction type.

9. System design

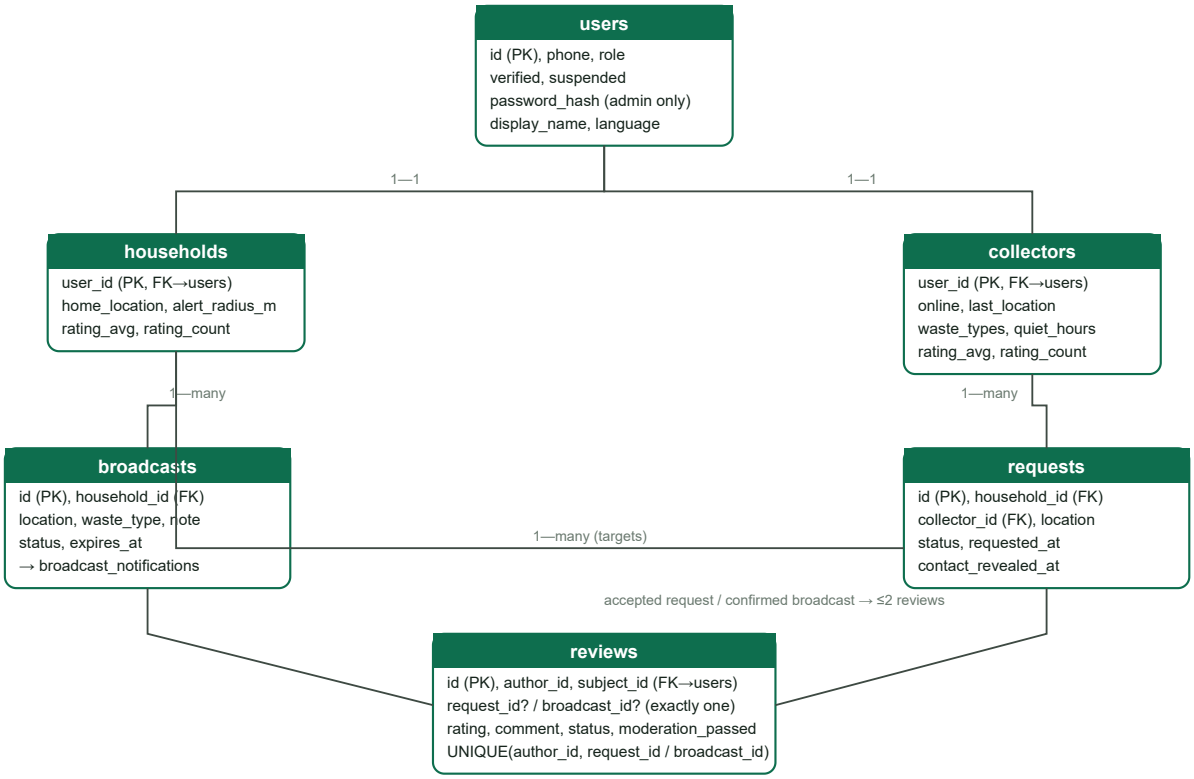
9.1 Architecture

One monorepo, one Postgres database, one Render Web Service serving both the built React app and the `/api` + Socket.IO endpoints from the same origin (no CORS, no second service to keep alive on a free tier):

Deliberate substitutions from the original design (each is a named technical-debt item, not an oversight): Leaflet+OSM raster tiles instead of MapLibre+vector tiles (TD-08); one Express service instead of separate household/collector/admin apps (TD-07); phone+password admin login instead of email+2FA (TD-06). The Redis hot-path/Postgres cold-path split and the BullMQ job queue (originally TD-05, a deferred substitution: `node-cron` sweeps directly against Postgres, no Redis) were **subsequently built as originally designed** once the user provisioned a real Upstash Redis instance — the diagram above reflects the as-built state, not the interim substitution. See `Technical_Debt_Plan.md` TD-05 for what was verified.

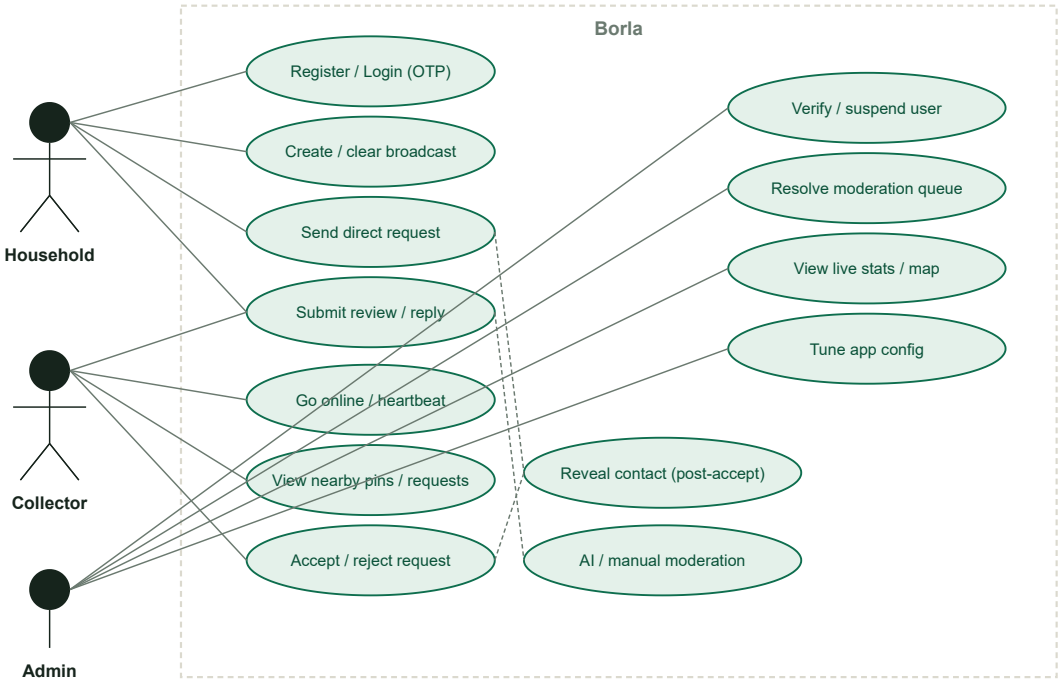
9.2 Data design (ER overview)

The full schema is `server/migrations/001_init.sql` (12 tables) plus `002_request_lifecycle.sql` (adds `requests.arrived_at` / `cancelled_by` and the `cancelled` status, for FR-28/FR-29 — a second, separately-tracked migration file rather than editing the first, since `001_init.sql` had already been applied in production; `server/src/db/migrate.ts` tracks applied files by name in `_migrations` so it only ever runs a new one once). Core entities and their relationships:



Supporting tables not pictured: `otp_codes` (OTP hashes/expiry), `broadcast_notifications` (who was notified, for "Did they come?" attribution), `pickup_confirmations`, `review_replies` (≤1 per review), `moderation_flags` (AI/user-report queue), `audit_log` (every admin action), `app_config` (live-tunable settings).

9.3 Use-case diagram



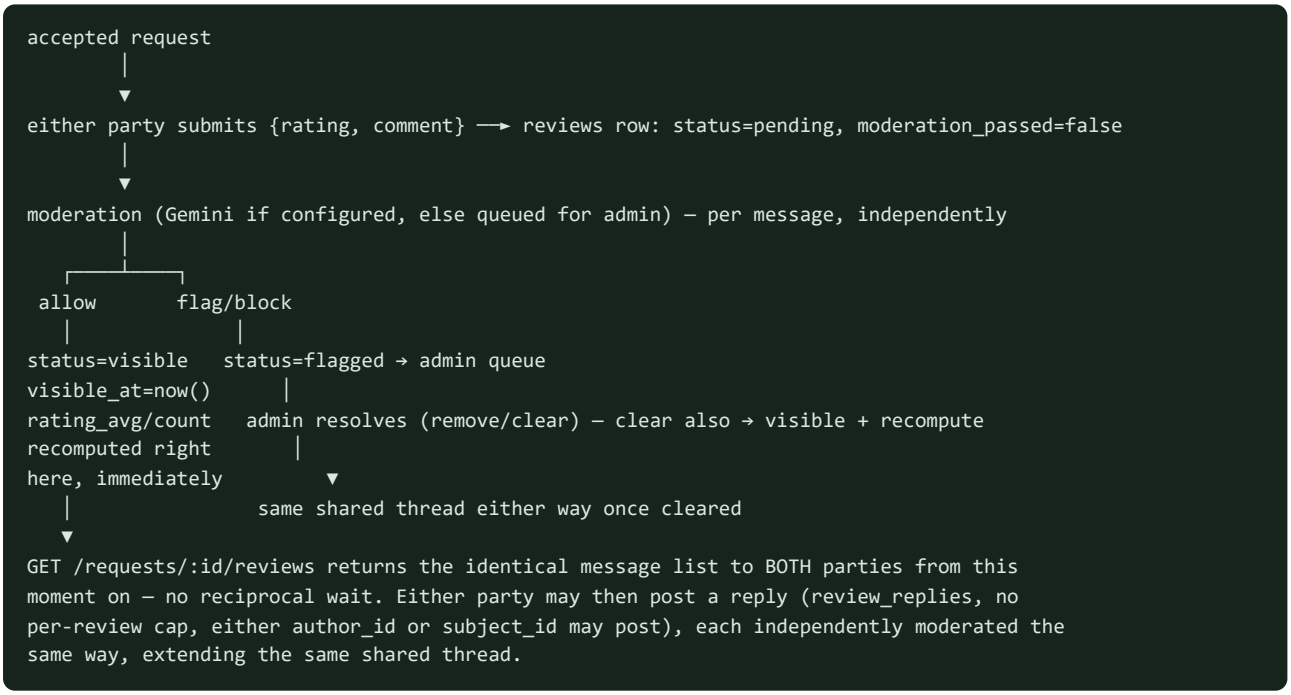
9.4 Sequence: broadcast fan-out (design §3.1/§7)

```
Household        Express API            Postgres/PostGIS        Collector(s)
    |  POST /broadcasts  |                      |                    |
    |------------------->|  INSERT broadcast    |                    |
    |                    |--------------------->|                    |
    |                    |  SELECT online       |                    |
    |                    |  collectors WHERE    |                    |
    |                    |  ST_DWithin(...)     |                    |
    |                    |--------------------->|                    |
    |                    |<---------------------| rows: candidates   |
    |                    |  filter: waste-type match, not quiet-hours |
    |                    |  INSERT broadcast_notifications (per match)|
    |                    |--------------------->|                    |
    |                    |  emit "broadcast:new" over Socket.IO ---------->|
    |  201 {broadcast,   |                      |                    |
    |<-----notifiedCount}|                      |                    |
```

9.5 State machine: request plane (design §3.2)

9.6 Activity: shared review thread per request (design §16, revised — TD-15)

The original design (and this build's first version) released reviews double-blind: hidden until *both* sides had reviewed, or a review window closed. User testing against the live app surfaced a real problem with that: the two parties to the same request could see different content on the same card depending on who had reviewed whom, which read as broken rather than intentional. TD-15 documents the decision to remove it in favour of one shared, symmetric, AI-moderated chat thread per request:



10. Implementation

10.1 Technology stack

Layer	Choice	Why
Server language/framework	Node.js + Express + TypeScript	Same language across stack; Express is small enough to reason about every middleware
Database	PostgreSQL 16 + PostGIS	Real geospatial queries (`ST_DWithin` , `ST_Distance`) without hand-rolled Haversine math
Realtime	Socket.IO + Redis adapter	JWT-authable, room-per-user, graceful fallback to polling; the Redis adapter makes cross-instance event delivery correct if scaled beyond one Node process
Cache/queue	Redis (Upstash) + BullMQ	Live presence/geo (`GEOSEARCH`), pin/rate-limit keys, and a real job queue (retries, backoff, repeatable schedulers) replacing the original `node-cron` sweeps — Technical_Debt_Plan.md TD-05
Frontend	React + Vite + TypeScript	Fast dev loop; one codebase serves all three roles via role-routing
Maps	Leaflet + OpenStreetMap raster tiles	Zero-config, no tile-provider account
Routing	OSRM public demo server (`router.project-osrm.org`)	Zero-config road route for FR-27's accepted-request route, same trade-off as the tile layer above; straight-line fallback if unreachable (Technical_Debt_Plan.md TD-14)
AI	Google Gemini API (optional)	Free tier for moderation; degrades gracefully without a key
SMS	GiantSMS (optional)	Real OTP delivery when configured; falls back safely (see Technical_Debt_Plan TD-02)
Auth	JWT (access+refresh) + bcrypt	Stateless API auth; OTP/password hashes never stored in plaintext
PWA	`vite-plugin-pwa` + Workbox	Installable app shell + explicit update-available prompt (resolves most of Technical_Debt_Plan TD-04)
Deployment	Render (one Web Service + one Postgres)	Single free-tier footprint, `render.yaml` blueprint

10.2 What was actually built

Every functional requirement tagged Must-Have or Should-Have in `SRS.md` §6 is implemented and exercised by the automated test suite: a **unified sign-in flow** where the phone number alone decides what happens next (a branded splash screen leads into a single phone-entry step; a seeded admin's number is auto-detected and routed to a password prompt; any other number gets an OTP, and only a genuinely new number is asked to pick a role and a name — an existing user logs straight in), with real SMS delivery attempted first; presence toggle/heartbeat with self-healing offline detection; the full broadcast plane (create/fan-out/clear/auto-expire); the full request plane (create/seen/accept/reject/auto-timeout with idempotent transitions); reveal-on-accept masked contact; two-sided reviews with real-interaction enforcement; AI-or-manual moderation, per message; an open reply thread (either party, unlimited messages — TD-15) shown identically to both sides on the request card the moment each message clears moderation, with rolling rating aggregates recomputed on every visibility change, not just at release time; the full admin console (verify/suspend/reinstate, moderation queue, audit log, live stats, live ops map, live config

tuning); a real installable PWA (manifest, icons, a Workbox service worker precaching the app shell, an explicit "Update available" prompt, and a native "Install app" button) — verified by checking `navigator.serviceWorker.getRegistrations()` against the actual production build, not just trusting the plugin; a Redis-backed hot path for presence/geo-matching/rate-limiting plus a real BullMQ job queue (broadcast fan-out and AI moderation both run as retryable background jobs rather than inline in the request handler) and a Socket.IO Redis adapter, since Technical_Debt_Plan.md TD-05 was resolved; FR-27, a collector-to-household / household-to-collector **route** once a direct request is accepted — a road-following route where the routing service resolves, a straight line otherwise, with distance/ETA — respecting the same reveal-on-accept timing as the phone number (TD-14 covers the third-party routing dependency this introduces); and a fuller **request lifecycle** (FR-28–FR-32): either party can **cancel** a request any time before arrival (a household previously had no way to withdraw one at all); the server itself detects a collector's **arrival** at the pickup point from the same position updates already used for presence (`ST_DWithin` against the request's stored location, no client self-reporting to trust) and notifies both sides in real time; resolved requests (arrived/cancelled/rejected/timed-out) move into a separate **History** tab so the active Requests view stays focused on what needs attention; a review and its reply now show **on the request they belong to**, not only in a flat Profile list; and the active-requests list sorts **closest-first by live route distance**, re-sorting as either side's position updates rather than only reflecting distance at load time.

10.3 Code organisation

```
server/src/modules/{auth,presence,broadcasts,requests,reviews,admin}/routes.ts   — one router per domain
server/src/redis/{client,presence,rateLimit}.ts                                  — Redis hot-path (TD-05)
server/src/jobs/{queues,scheduler,workers,index}.ts                              — BullMQ queues/schedulers/wo
server/src/ai/moderation.ts, server/src/utils/{otp,sms,jwt,quietHours,phone}.ts  — isolated, unit-testable lo
client/src/pages/{Login,HouseholdHome,CollectorHome,AdminDashboard,Profile}.tsx  — one screen per role/concer
client/src/components/{MapView,RoutePanel,RequestReviews,ProtectedRoute}.tsx     — shared, reusable (RequestR
```

10.4 Security controls actually implemented

JWT-gated routes with role middleware; bcrypt-hashed OTP codes and admin passwords; parameterised SQL everywhere (no injection surface); a per-IP rate limiter on all `/api/` traffic plus Redis-backed, per-phone (`ratelimit:otp:{phone}`) and per-collector (`ratelimit:notif:{id}`) caps matching the original design's own rate-limit keys (TD-05); ownership checks on every mutating route (a household can only clear *its own* broadcast, etc.); phone numbers normalised to one canonical form before every lookup/insert (`utils/phone.ts` , D-07) and stripped from API responses until a request is accepted; fail-closed moderation (no verdict ⇒ stays hidden).

11. Testing (summary)

57/57 automated tests passing (54 server — unit + Supertest integration against a real PostgreSQL+PostGIS instance *and* a real Redis instance; 3 client — React Testing Library, down from 5 after `ReviewForm.tsx` /its test were retired along with the double-blind model, TD-15) at time of submission, plus a scripted manual system/UAT pass and a security/usability review. Thirteen real defects were caught and fixed during development — eight in the automated suite (a broken first-time-signup code path, a review-reply status gap, a rating-aggregate staleness bug found by reasoning through every path that touches a review's visibility (D-11), and five others), one in a scripted screenshot pass (missing avatar CSS + broken initials logic, D-09), one found by watching a just-shipped fix operate for real in production (a moderation-retry gap that meant a transient AI failure got exactly one attempt forever, D-13), and three found only by treating the *live deployed app or its logs* as the actual object under test: the admin account was reachable via the weaker OTP flow, bypassing its intended phone+password requirement entirely (found by me, re-testing production); an already-registered phone number typed without its leading `+` was treated

as brand-new instead of logging straight in (found by the user, D-07); and a hardcoded Gemini model ID started 404ing the moment a real key went live in production (D-08) — plus, adjacent to the SMS defect, the two arbitrary seeded demo phone numbers would have silently "succeeded" into a gateway with no phone behind them, locking any examiner out of the graded accounts. Full detail, every test case, and all thirteen defect write-ups are in `Testing_Report.md`.

12. Technical debt

Sixteen tracked items (`Technical_Debt_Plan.md`), each with Debt→Cause→Impact→Priority→ Resolution. One is 🔴 Critical (admin has no 2FA), six are 🟡 Scheduled — including the GiantSMS OTP integration, which was **confirmed live in production** (the gateway accepted a real send request end-to-end) but not yet confirmed to a real handset — and the rest are 🟢 Acceptable/Resolved, including TD-01 (AI moderation) and TD-05 (Redis/BullMQ), both confirmed working end-to-end rather than just configured, and TD-15, the explicit, reasoned decision to drop double-blind review release in favour of a shared, symmetric review thread (§9.6, both prompted by direct user feedback on the live app). The single largest remaining item is the missing native background-location collector app (TD-03) — the original design's own #1 risk — deliberately left as the biggest future-evolution item rather than attempted unsafely inside the exam window. Full register, priorities, and a phased repayment plan are in `Technical_Debt_Plan.md`.

13. Deployment

Deployed to **Render**: one free-tier Web Service (`buildCommand: npm ci --include=dev && npm run build`, `startCommand: npm run start -w server`) serving both the compiled API and the built React client from one origin, plus one free-tier managed Postgres with the PostGIS extension enabled by the first migration. `render.yaml` in the repository root fully describes the topology (Infrastructure-as-Code, not click-ops) including which secrets (`GEMINI_API_KEY`, `GIANTSMS_API_TOKEN`, `GIANTSMS_SENDER_ID`) must be set manually in the dashboard (`sync: false`) versus generated automatically (`JWT_*_SECRET`, `generateValue: true`) or wired from the managed database (`DATABASE_URL`, `fromDatabase`). See `Deployment_and_Source_Links.txt` for the live URL and credentials, and `User_Manual.md` §6 for exact redeploy/rollback steps.

14. User manual (summary)

Full walkthroughs for all three roles — including how to read the on-screen OTP fallback if SMS delivery isn't available — are in `User_Manual.md`. In short: sign-in is a single phone-number entry point on the `/login` screen; an already-registered number logs straight in, a genuinely new number is asked for a role (household/collector) and a name only after the OTP is verified, and a seeded admin's number is auto-detected and routed to a password prompt instead of an OTP — no manual role/admin toggle anywhere. Once in: a household broadcasts with one tap or searches nearby collectors to request directly; a collector toggles online and responds to what appears; an admin lands on `/admin` after the password step.

15. Maintenance strategy

Type	Approach
Corrective	GitHub issue → reproduce with an integration test that fails → fix → test passes → deploy. The existing 54 server tests are the regression net — this is exactly how D-07 (phone normalisation), D-11 (rating-

Type	Approach
	aggregate staleness), and D-13 (moderation-retry gap) were all closed, with a failing test added before each fix.
Adaptive	Config changes (radius, TTLs, timeouts) go through `app_config` and the admin UI — no redeploy needed for the most likely "the environment changed" adjustments.
Perfective	Tracked as the technical-debt repayment plan (`Technical_Debt_Plan.md` §4) — i18n, offline shell, deeper test coverage.
Preventive	`npm audit` run before any dependency bump; the health-check endpoint (`/api/health`) lets Render auto-restart a wedged instance.
Security updates	Dependencies pinned with caret ranges; `npm audit` reviewed monthly if the project continues; JWT secrets rotated by regenerating the Render env vars (stateless tokens, no migration needed).
Dependency updates	Renovate/Dependabot recommended once this leaves the exam context — not configured now, itself a minor debt item.
Performance	PostGIS GIST index already in place for the two hot geo-queries; would add a read replica for the admin analytics reads before scaling further (per the original design's own §17 guidance).
Scalability	The single-instance ceiling is already removed at the code level — Redis-backed presence/rate-limiting, BullMQ jobs, and a Socket.IO Redis adapter are all in place (TD-05 resolved); the remaining step is operational (add a second Render instance and confirm behaviour under real concurrent load), not architectural.
New features / user feedback	Photo→waste-type classification (TD-11) and native background location (TD-03) are the two most-requested-shaped gaps based on the original design's own accessibility and reliability goals.
Technology changes	Node/Postgres/React are all mainstream, actively maintained — no forced-migration risk foreseen in the near term.

16. Future evolution

Phased roughly as the original design's own roadmap (`borla-technical-design.md` §15), adjusted for what this build already covers:

- **v1.1**: real SMS delivery confirmed end-to-end (finish TD-02), admin 2FA (TD-06).
- **v1.2**: Gemini key provisioned in production, CI pipeline on every push (TD-01, TD-13).
- **v1.3**: native Capacitor collector app with background geolocation (TD-03) — tested on real low-end Android hardware, per the original design's own risk mitigation.
- **v2.0**: i18n in Twi/Ga (TD-09), offline write queue to finish the PWA shell (TD-04), photo→waste-type classification (TD-11). (Redis + BullMQ + Socket.IO Redis adapter, originally slated for this phase as TD-05, was built ahead of schedule — see §9.1/§10.1.)
- **Beyond**: the original design's own explicitly-deferred set — in-app payments/escrow, live turn-by-turn tracking, auto-fallback dispatch, provider call-masking, and expansion beyond a single pilot neighbourhood.

17. Limitations

This is a web-only build: collectors only report position while their browser tab is open and foregrounded (no native background service — TD-03), which the original design itself identifies as the single biggest reliability risk for a real deployment. OTP delivery via GiantSMS is confirmed live in production at the HTTP layer but not yet confirmed to a real handset (TD-02). Admin authentication has no 2FA (TD-06). Test coverage is broad across every module but

not exhaustive within any one of them (TD-10). The system has only ever run against a handful of seeded/manually-created accounts — it has not been load-tested.

18. Conclusion

Borla demonstrates that a genuinely two-sided, stateful/stateless-hybrid matching platform with a real trust layer and admin console can be specified, estimated, designed, built, tested, documented, and deployed inside a hard time-box — provided the estimation step is used honestly to *cut scope*, not to justify skipping it. The most valuable engineering decision in this build was not any single line of code but the early recognition (via UCP) that the original design was 16–80× too large for the window available, which forced a breadth-first, thin-vertical-slice strategy instead of a narrower, more polished one. What that strategy left unfinished is not hidden — it is named, prioritised, and scheduled in `Technical_Debt_Plan.md`.

19. References

- Course specification: *CSCD 602 Advanced Software Engineering Project Exams.pdf*, University of Ghana, Department of Computer Science (Examiner: Prof. Solomon Mensah).
- Original architectural design: `borla-technical-design.md` (authored ahead of this build).
- UI design system: `borla_UI_design.html` (colour tokens, typography, and component patterns reused directly in `client/src/styles/`).
- Karner, G. (1993). *Resource Estimation for Objectory Projects* — Use Case Points method.
- Express — https://expressjs.com · PostgreSQL/PostGIS — https://postgis.net · Socket.IO — https://socket.io · React/Vite — https://vitejs.dev · Leaflet/OpenStreetMap — https://leafletjs.com, https://www.openstreetmap.org · BullMQ — https://docs.bullmq.io · Redis/Upstash — https://upstash.com/docs/redis · Google Gemini API — https://ai.google.dev · GiantSMS — https://giantsms.com/developer · OSRM (public routing demo server) — https://project-osrm.org · Render — https://render.com/docs.